Experiment No 7

Course Outcome: CO4 **Blooms Level: L3**

Aim: Implement Outlier detection using a density-based method.

Abstract: Real-world datasets often contain noise and outliers that affect the accuracy of clustering and data analysis. Traditional clustering algorithms are not always effective in detecting clusters of arbitrary shapes or distinguishing noise points. Therefore, there is a need for a density-based clustering approach that can identify dense regions in a dataset and detect outliers effectively. Apply the DBSCAN algorithm using the Python sklearn library to analyze the dataset based on parameters such as epsilon (ε) and minimum points (MinPts). Classify the data points as core, border, or noise based on neighborhood density, and visualize the identified clusters and outliers using the matplotlib library to better interpret the data distribution.

Theory :- Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is a base algorithm for density-based clustering. It can discover clusters of different shapes and sizes from a large amount of data, which contains noise and outliers.

The DBSCAN algorithm uses two parameters:

minPts: The minimum number of points (a threshold) clustered together for a region to be considered dense. For example, if we set the minPoints parameter as 5, then we need at least 5 points to form a dense region.

eps (ε): A distance measure that will be used to locate the points in the neighborhood of any point. specifies how close points should be to each other to be considered a part of a cluster. It means that if the distance between two points is lower or equal to this value (eps), these points are considered neighbors.

There are three types of points after the DBSCAN clustering is complete:

Core — This is a point that has at least m points within distance n from itself.

Border — This is a point that has at least one Core point at a distance n.

Noise — This is a point that is neither a Core nor a Border. And it has less than m points within distance n from itself.

The algorithm proceeds by arbitrarily picking up a point in the dataset (until all points have been visited). If there are at least 'minPoint' points within a radius of 'ε' to the point then we consider all these points to be part of the same cluster.The clusters are then expanded by recursively repeating the neighborhood calculation for each neighboring point

DBSCAN algorithm can be abstracted in the following steps:

- Find all the neighbor points within eps and identify the core points or visited with more than MinPts neighbors.
- For each core point if it is not already assigned to a cluster, create a new cluster.
- Find recursively all its density connected points and assign them to the same cluster as the core point.

- A point a and b are said to be density connected if there exist a point c which has a sufficient number of points in its neighbors and both the points a and b are within the eps distance. This is a chaining process. So, if b is neighbor of c, c is neighbor of d, d is neighbor of e, which in turn is neighbor of a implies that b is neighbor of a.
- Iterate through the remaining unvisited points in the dataset. Those points that do not belong to any cluster are noise.

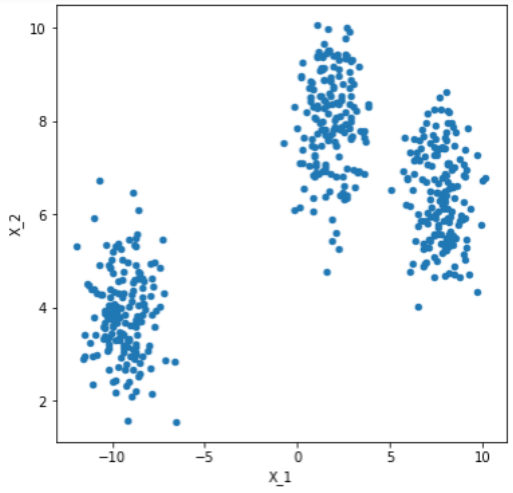
Program:

GENERATE CLUSTERING DATA

In [ ]:
```
import numpy as np
from sklearn.datasets import make_blobs
from matplotlib import pyplot as plt
from matplotlib.pyplot import figure
from pandas import DataFrame
X,_=make_blobs(n_samples=500, centers=3, n_features=2, random_state=20)
```

VISUALIZATION OF DATA

In [ ]:
```
df=DataFrame(dict(x=X[:,0],y=X[:,1]))
fig, ax=plt.subplots(figsize=(6,6))
df.plot(ax=ax,kind='scatter',x='x',y='y')
plt.xlabel('X_1')
plt.ylabel('X_2')
plt.show()
```



In [ ]:
```
from sklearn.cluster import DBSCAN
clustering=DBSCAN(eps=1, min_samples=5).fit(X)
cluster=clustering.labels_
len(set(cluster))
```

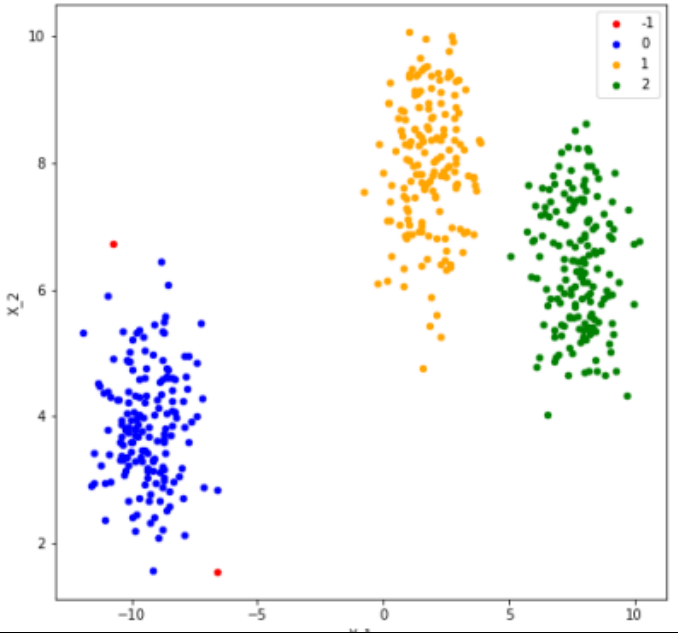
Out[ ]: 4

In [ ]:
```
clustering.labels_
```

Out[ ]:
```
array([ 0,  1,  0,  0,  1,  1,  2,  2,  0,  0,  0,  1,  2,  2,  2,  0,  1,
        0,  2,  2,  0,  0,  1,  2,  1,  1,  0,  1,  0,  2,  2,  2,  1,  2,
        0,  1,  0,  2,  1,  1,  2,  1,  2,  2,  2,  2,  0,  0,  0,  2,  2,
        2,  1,  1,  1,  0,  2,  1,  2,  2,  0,  2,  2,  1,  2,  0,  0,  2,
        0,  2,  2,  1,  2,  2,  1,  0,  0,  0,  2,  0,  1,  1,  0,  0,  0,
        2,  2,  2,  0,  2,  0,  2,  2,  0,  0,  2,  1,  0,  2,  0,  1,  2,
        1,  0,  2,  2,  2,  1,  1,  0,  0, -1,  0,  0,  0,  1,  1,  0,  1,
        2,  1,  2,  2,  0,  0,  2,  1,  2,  1,  2,  0,  2,  1,  0,  1,  1,
        2,  0,  2,  1,  1,  2,  2,  1,  1,  1,  2,  0,  2,  1,  0,  0,  0,
        0,  0,  1,  2,  0,  1,  0,  2,  0,  0,  2,  0,  0,  0,  0,  0,  0,
        2,  0,  1,  1,  2,  2,  2,  2,  0,  1,  1,  1,  1,  2,  0,  2,  1,
        2,  1,  2,  1,  1,  0,  1,  1,  1,  2,  2,  1,  1,  0,  2,  2,  1,
        0,  2,  2,  1,  2,  1,  0,  0,  1,  1,  0,  2,  1,  1,  0,  2,  0,
        2,  0,  2,  0,  2,  1,  0,  1,  0,  1,  0,  2,  0,  0,  0,  2,  1,
        0,  1,  1,  2,  2,  1,  2,  2,  2,  0,  1,  2,  2,  1,  0,  0,  2,
        2,  2,  1,  1,  0,  2,  1,  2,  2,  2,  0,  0,  2,  1,  0,  1,  1,
        2,  1,  0,  1,  0,  0,  1,  1,  1,  2,  1,  0,  2,  2,  2,  0,  2,
        0,  2,  0,  1,  1,  0,  2,  2,  0,  1,  2,  1,  0,  1,  0,  0,  0,
        1,  1,  0,  0,  0,  1,  1,  1,  2,  1,  1,  2,  2,  1,  2,  1,  1,
        2,  0,  1,  2,  2,  1,  2,  1,  2,  1,  1,  0,  0,  0,  2,  1,  1,
        0,  0,  2,  0,  0,  1,  2,  1, -1,  1,  0,  2,  1,  1,  0,  1,  0,
        1,  1,  0,  1,  2,  1,  1,  0,  0,  0,  2,  0,  2,  1,  1,  1,  0,
        0,  2,  2,  0,  2,  2,  1,  1,  2,  1,  1,  1,  1,  1,  2,  1,  2,
        2,  0,  1,  2,  0,  1,  0,  0,  2,  0,  1,  1,  2,  1,  2,  0,  1,
        2,  2,  0,  0,  0,  1,  2,  0,  0,  1,  1,  2,  1,  2,  1,  2,  2,
        2,  2,  0,  0,  0,  1,  1,  0,  0,  1,  1,  0,  2,  1,  2,  1,  1,
        1,  1,  2,  0,  1,  2,  1,  1,  1,  0,  0,  2,  0,  2,  0,  1,  1,
        0,  0,  0,  0,  0,  2,  2,  1,  2,  0,  2,  0,  1,  1,  0,  1,  0,
        0,  2,  2,  1,  2,  2,  0,  2,  0,  2,  2,  0,  0,  0,  0,  1,  1,
        2,  1,  0,  2,  2,  2,  1])
```

In [ ]:
```
def show_clusters(X,cluster):
  df=DataFrame(dict(x=X[:,0],y=X[:,1], label=cluster))
  colors={-1:'red',0:'blue',1:'orange',2:'green',3:'pink'}
  fig, ax=plt.subplots(figsize=(8,8))
  grouped=df.groupby('label')
  for key,group in grouped:
    group.plot(ax=ax,kind='scatter',x='x',y='y', label=key, color=colors[key])
  plt.xlabel('X_1')
  plt.ylabel('X_2')
  plt.show()
```

In [ ]:
```
show_clusters(X, cluster)
```



Conclusion: In this experiment, the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm was applied to perform density-based clustering and outlier detection. The algorithm groups data points based on the density of their neighborhood using parameters such as epsilon (ε) and minimum points (MinPts). It successfully identifies clusters of arbitrary shapes and classifies points as core, border, or noise, enabling effective detection of outliers in the dataset. The implementation using the Python sklearn library and visualization with matplotlib helps in understanding the clustering structure and distinguishing normal data points from noise.

Exercise 1:

Abstract: Apply the DBSCAN density-based clustering algorithm on the Iris dataset to analyze the distribution of data points. Identify dense regions that form clusters and detect possible outliers in the dataset based on the parameters epsilon (ε) and minimum points (MinPts). Visualize the resulting clusters and noise points using appropriate Python libraries.

Dataset:

UCI Repository link: https://archive.ics.uci.edu/ml/datasets/iris

.

Exercise 2:

Abstract: Use the DBSCAN algorithm to perform density-based clustering on the Breast Cancer Wisconsin dataset. Analyze the dataset to identify clusters of similar data points and detect observations that behave as outliers or noise. Visualize the clusters and detected outliers to understand the density distribution of the data.

Dataset: https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic)